TradeIO

D2 Detailed Design

Trading Journal and Behavioral Analytics Platform

Senior Design

Assignment #4: Design Diagrams · Design Stage: D2

Team Members

Ishaa Jain

Samarth Prajapati

October 2026

Project Goal

TradeIO is a trading journal designed to help traders record trading activity, reflect on execution and psychology, monitor risk and discipline, and receive useful performance and behavioral insights.

Contents

1 Header, Conventions, and D2 Overview **2**

2 Finished Components — D2 Diagrams **2**

2.1 D0 Coverage Audit . . . 2
2.2 C1 — User Interface . . . 3
2.3 C2 — Application/API Layer . . . 3
2.4 C3 — Trade and Journal Processing . . . 4
2.5 C4 — Risk and Behavior Analysis (D1 Completion) . . . 4
2.6 C5 — Data Storage (D1 Completion) . . . 4
2.7 E1 — Identity Provider / Authentication Glue . . . 5
2.8 E2 — Optional AI Insight Service . . . 5
2.9 E3 — Notification Service / Delivery Glue . . . 5
2.10 Cross-cutting Glue and Proposed Extensions . . . 5
2.11 D2 Error-Path Diagram . . . 6
2.12 Accountability Block State Machine (Proposed) . . . 7

3 Edge Cases and Error Handling **7**

4 Requirements Traceability Table **9**

5 Design Rationale **11**

6 Peer Design Review Notes **13**

6.1 Feedback Received . . . 13
6.2 Peer Review Action Items . . . 13
6.3 Summary of Feedback . . . 13
6.4 Issues We Raised While Reviewing Another Team . . . 13

1. Header, Conventions, and D2 Overview

Project title: TradeIO — Trading Journal and Behavioral Analytics Platform.
Goal: TradeIO is a trading journal designed to help traders record trading activity, reflect on execution and psychology, monitor risk and discipline, and receive useful performance and behavioral insights.

Two-minute overview. TradeIO enables the trader to document their transactions and reflections, define personal risk guidelines, and perform deterministic and behavioral feedback on performance. D0 has classified five components (C1 to C5), the external identity, insight, and notification (E1 to E3), and the interfaces I1 to I10. D1 has detailed the relational data model in C5 and the risk and psychological algorithms in C4, while defining initial API and technology decisions. D2 finalizes the details of C1 to C3, details the integrations and local glue needed for support, documents failures through the architecture, and maps the requirements proposed to the interfaces.

Diagram conventions. Rounded rectangles that are white are either processing steps/states for TradeIO, while shaded rectangles are failure or terminal states. Diamonds depict conditions on which branches can be made, while the solid arrows depict the possible course of execution and have labels if a decision was made. The dashed rectangle outside is a provider and is not inside the implementation boundary of TradeIO. All diagrams have the title and goal above them.

Known gaps and open questions (not implied to be resolved): (G1) The uploaded D0 does not include a broker as a fourth external system although later requirements describe optional brokerage connectivity; confirm whether the consolidated PDF should amend D0. (G2) D1's TRADE table is single-entry/single-exit, whereas multi-fill trading needs child records; confirm MVP depth. (G3) D1 distinguishes user time zone and account currency, but does not establish FX conversion rules: do not aggregate across currencies without a validated conversion policy. (G4) Whether an accountability block restricts login to TradeIO, trade entry only, or both requires agreement; it cannot block activity at an external broker. (G5) Selection and operational limits of AI, email and broker providers remain undecided. (G6) Actual peer-review results are pending and are not fabricated in this draft. (G7) Original approved user-story IDs must be reconciled against the team's final User_Stories file.
Draft status: proposed behaviors, risk controls and prioritization in this D2 are review candidates, not claims of completed implementation or verified team agreement.

2. Finished Components — D2 Diagrams

2.1 D0 Coverage Audit

ID	D1 coverage	D2 remaining specification / reference
C1 UI	Framework and accessibility choice	View state, forms, behavior and contracts in §2.2
C2 API	Selected v1 endpoints	Authentication, permissions, error envelope and orchestration in §2.3
C3 Processing	Mentioned in D0	Normalization and persistence workflow in §2.4
C4 Analysis	D1 algorithms A/B	Review of limits, asynchronous failure and contract boundaries in §2.5
C5 Storage	D1 relational ER model	Transactions, ownership, idempotency and optional new tables in §2.6

ID	D1 coverage	D2 remaining specification / reference
E1 Identity	Clerk selected	Session/authorization adapter in §2.7
E2 AI	Optional service selected	Adapter, redaction and degraded modes in §2.8
E3 Notification	Provider unspecified	Outbox and failure recording in §2.9
Glue	Scattered responsibilities	Settings, audit logging, broker adapter and jobs in §2.10

2.2 C1 — User Interface

Data model: No persistent UI-owned data. Client-side state: `FormState{draft, validationErrors, submitState}` and `ViewState{loading,ready,empty,error,stale}`. Saved trade drafts are *not* assumed to survive a page reload.

Behavior: (1) Check session before showing protected views. (2) Load account-scoped data. (3) Validate mandatory fields on input; server re-validates. (4) Submit with one idempotency key and disable duplicate submission until a response arrives. (5) Display saved trade only after server confirmation. (6) Display analysis age and a retry option if analysis is unavailable.

Interface: `submitTrade( TradeEntryDTO, idempotencyKey): Promise<TradeResult>`; `loadDashboard(accountId, period): Promise<DashboardDTO>`; I1, I7, I9. Error Codes: 400 Field Validation Error, 401 Login Required, 403 Not Authorized, 409 Conflict or Active Block, 429 Rate Limited, 503 Service Unavailable. Keyboard and screen reader feedback is necessary; never rely on color alone for communication.

2.3 C2 — Application/API Layer

Data model: No unique business table. Uses `User`, `TradingAccount`, `RiskSettings`, and proposed `AuditEvent` for recorded authorization-sensitive actions.

Behavior: Session validation; extract user ID from server claims (no body should be sent); validate all fields and user ownership; validate active block under policy; execute core mutation within database transaction; enqueue any side effect tasks after transaction commit; return typed response with request ID. If 401/403/409 error occurs, don't make the trade.

Interfaces: Keep D1's `/api/v1/trades`, `/api/v1/journal`, `/api/v1/accounts/{id}/analysis`, `/api/v1/accounts/{id}/risk-settings`. Proposed additions: `GET /api/v1/accounts/{id}/dashboard`, `PUT /api/v1/profile/preferences`, `POST /api/v1/accounts/{id}/block`, `GET /api/v1/notifications`. Error envelope shown below. More complex calls pass one typed object, not a long list of scalar parameters.

```
{ "error": { "code": "RISK_LIMIT_REACHED",
  "message": "A configured limit was reached.",
  "requestId": "request-uuid", "recoverable": false,
  "fieldErrors": {} } }
```

Use codes 400 Invalid field, 401 Missing session, 403 Unauthorized, 404 Not Found within owned scope, 409 Conflicting state, 429 Throttling, 502 Upstream invalid response, 503 Unavailable and 500 Unexpected internal failure. Always avoid leaking internal stack traces.

2.4 C3 — Trade and Journal Processing

Data model: D1 `TRADE` and `JOURNAL_ENTRY`; proposed `TRADE_FILL(fill_id,trade_id,side,quantity,price,fee,executed_at)` provided that partial entry and exit exist in MVP. A normal DTO will have Account ID, Instrument, Side, Timestamps, Price/Quantity as decimals in strings, and Journal.

Behavior: Normalization to UTC for enumerations and timestamps; verification of quantity/monetary calculations using decimals; ownership validation between the journal and trade; detection of duplicate imports/manual attempts using an idempotency key; atomic commitment; calculation of C4 post-persistence. D1 contains one entry and an optional exit per trade; the fill child is a new addition and not part of existing D1 data.

Interface: `processTrade(ValidatedTradeInput): Result<StoredTrade,DomainError>` (I2/I3); `processJournal(ValidatedJournalInput)`. Exclude non-positive values, invalid lifecycle sequence, invalid time stamp or foreign trading account number. Differentiate between validation and storage issues. **Formula choice:** Assume that `priceDifference * quantity * leverage` formula may not apply to all instruments. The calculation of P&L depends on the contract multiplier of the instrument and other factors such as units and fees; an asset-specific formula is required.

2.5 C4 — Risk and Behavior Analysis (D1 Completion)

Data model: D1 `RISK_SETTINGS` and `ANALYSIS_RECORD`; proposed `RISK_EVENT(event_id,account_id,type,occurred_at,rule_version)` to preserve alert history.

Behavior: Keep D1 Algorithm A (timezone-aware daily/weekly limits) and B (deterministic psychology) as the source of truth. Compare thresholds exactly, include realized P&L and open trades separately, calculate zero-trade summary without division by zero and do not infer psychological score that was never stored. In case of concurrent trade placement, the definitive count of trades needs to be verified in account lock scope or serializable transaction.

Interface: `evaluateRisk({accountId,asOf,rules}): RiskEvaluation` (I4/I5); `summarizeBehavior({accountId,start,end}): BehaviorSummary`. Errors: invalid range (400), stale metrics (explicit freshness status), missing data (empty summary), database failure (503). Risk warning vs. submission blocking is a separate product policy requiring [**TEAM TO CONFIRM**].

2.6 C5 — Data Storage (D1 Completion)

Data model: Reuse D1 `USER`, `TRADING_ACCOUNT`, `TRADE`, `JOURNAL_ENTRY`, `RISK_SETTINGS`, `ANALYSIS_RECORD`, `GOAL`. Optional D2 additions are documented separately below.

Behavior: Account-scoped queries use user ownership constraints. Transactions commit trade writes atomically. Enforce unique request/import keys to avoid duplicates; foreign keys reject missing account/trade references. Only published analysis snapshots are displayed as current if derived from an identified input revision. Retry transient connections with a bound policy; never blindly replay a non-idempotent write.

Interface: `withTransaction(fn)`, `saveTrade(dto,key)`, `getAccountTrades(accountId,range)`, `saveAnalysis(accountId,revision,result)`. Errors: unique violation mapped to 409 or existing result, foreign key violation 400/404, connection failure 503, exhausted retries 503.

2.7 E1 — Identity Provider / Authentication Glue

Data model: Clerk owns credential/session details. TradeIO retains D1 `USER` keyed to a verified external subject; the key mapping must be unique.

Behavior: Validate Clerk session; map subject to local user; authorize each account/resource; distinguish 401 unauthenticated from 403 authenticated but forbidden. On provider disruption, reject new protected mutations rather than trusting unverified identities.

Interface: `requireUser(session): AuthPrincipal` (D0 I6); `assertAccountOwner(userId,accountId)`. Failures: expired token 401, ownership 403, provider outage 503.

2.8 E2 — Optional AI Insight Service

Data model: No provider-owned data in TradeIO's scope. Proposed `INSIGHT_RUN(run_id,account_id, period,status,generated_at,summary)` tracks non-sensitive output and job status.

Behavior: Send only period-aggregated, minimized statistics to the adapter; do not send private free-form journal text by default. Apply timeout and rate limits; validate output shape; use deterministic C4 summary if the provider fails. Keep AI statements explicitly advisory, not financial instructions.

Interface: `generateInsight(InsightContext): Result<InsightResult,ProviderError>` (I8). Errors: 429 queue/defer, timeout retry within bounded budget, malformed output discard, service outage show unavailable. All recoverable without affecting saved trades.

2.9 E3 — Notification Service / Delivery Glue

Data model: Proposed `NOTIFICATION_OUTBOX(id,user_id,event_type,payload,status,attempts, next_attempt_at,created_at)` and `NOTIFICATION_PREFERENCE(user_id,channel,enabled)`.

Behavior: In-app event logging should precede delivery request to the third party; send one unique provider notification message after successful core transaction commit; retry transient failures using bounded exponential backoff algorithm; consider failure final and log in-app alert. Delivery confirmation does not indicate receipt by user.

Interface: `enqueueNotification(event): eventId`; `send(event): DeliveryResult` (I10). Errors: disabled channel -> no external send; 429/5xx -> retry; malformed destination -> failed with actionable status; provider timeout -> unknown delivery state until reconciled.

2.10 Cross-cutting Glue and Proposed Extensions

Configuration: Existing D1 `USER.time_zone`, `USER.base_currency`, `TRADING_ACCOUNT.currency`, and `RISK_SETTINGS` remain authoritative. `updatePreferences(PreferencesDTO)` validates IANA time zone and ISO-4217 codes (I9); reject invalid values (400) and conflicting edits (409). Save effective change time and preserve historical UTC instants. Do not silently translate recorded amounts between INR and USD.
Logging and audit: Proposed `AUDIT_EVENT(id,actor_id,account_id,action,outcome,at,request _id)`. `recordAudit(event)` logs risk overrides, blocks, permission denials and settings changes without full journal contents, access tokens or financial credentials. Storage failures must not reveal secrets; high-risk control changes should fail closed if an audit requirement is adopted.
Broker import (proposed): `BROKER_CONNECTION(id,account_id,provider,external_account_ref, status,last_sync_at)`; encrypted access secrets must use provider/managed secret storage, not plaintext records. `syncTrades(connectionId,cursor)` pages through provider history, normalizes currency and

instrument units, and upserts by provider+external-trade-ID. Errors: revoked access -> reconnect; throttling -> deferred retry; unsupported provider -> manual entry remains available; missing contract multiplier -> mark unverified instead of inventing P&L. This broker integration is not shown in the supplied D0 figure (G1).

Accountability block (proposed): `ACCOUNTABILITY_LINK(trader_id,partner_id,status)` and `ACCOUNT_BLOCK(block_id,trader_id,partner_id,starts_at,ends_at,status)`. Only an accepted partner may start a 10-minute to 24-hour TradeIO-specific block. The server verifies the active block on the protected operation; expiry is evaluated from UTC server time, not only a scheduled job. When no partner is linked, no block controls are exposed. This cannot suspend external brokerage trading. Blocking scope needs team decision (G4).

2.11 D2 Error-Path Diagram

TradeIO — D2 Trade Submission and Recovery

Goal: TradeIO is a trading journal designed to help traders record trading activity, reflect on execution and psychology, monitor risk and discipline, and receive useful performance and behavioral insights.

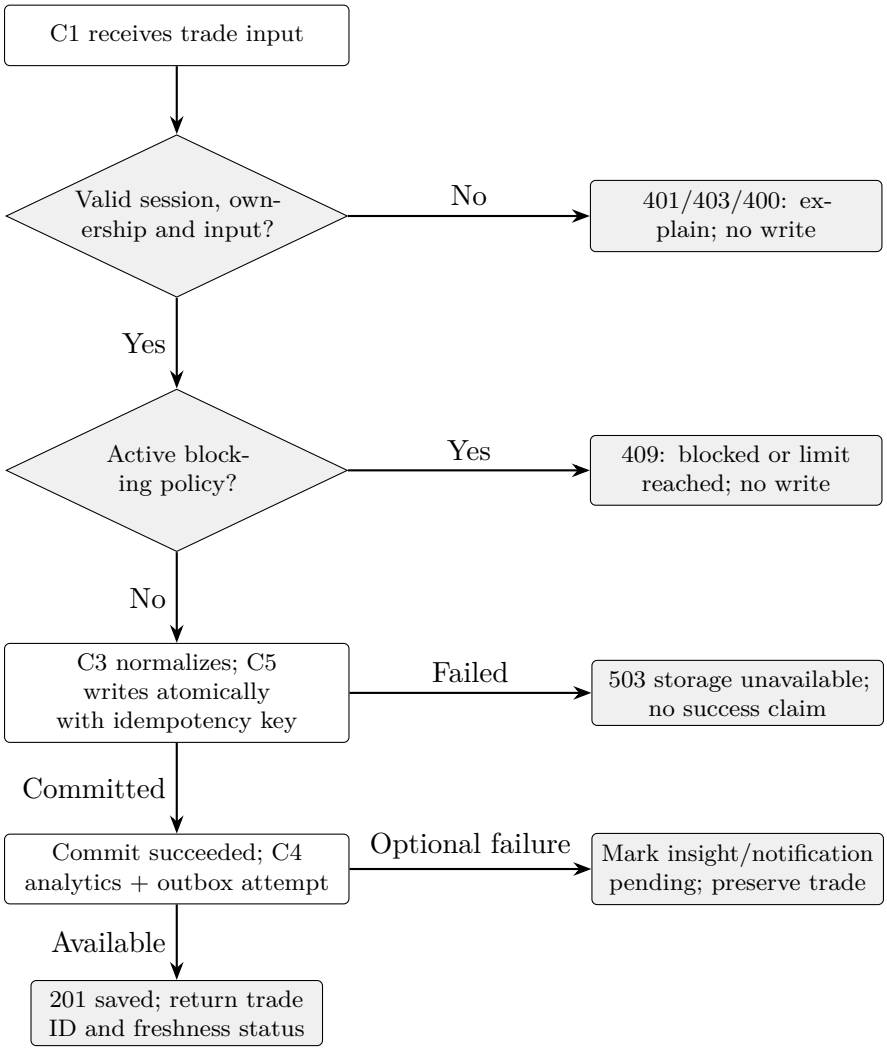


Figure 1: Transaction boundary and explicit error outcomes. The block decision is proposed and subject to G4.

Figure legend: white rounded box = TradeIO activity; diamond = yes/no decision; shaded box = response or degraded outcome; solid arrow = control transfer; arrow label = outcome. Failure paths do not create a false saved result.

2.12 Accountability Block State Machine (Proposed)

TradeIO — D2 Accountability State Machine

Goal: TradeIO is a trading journal designed to help traders record trading activity, reflect on execution and psychology, monitor risk and discipline, and receive useful performance and behavioral insights.

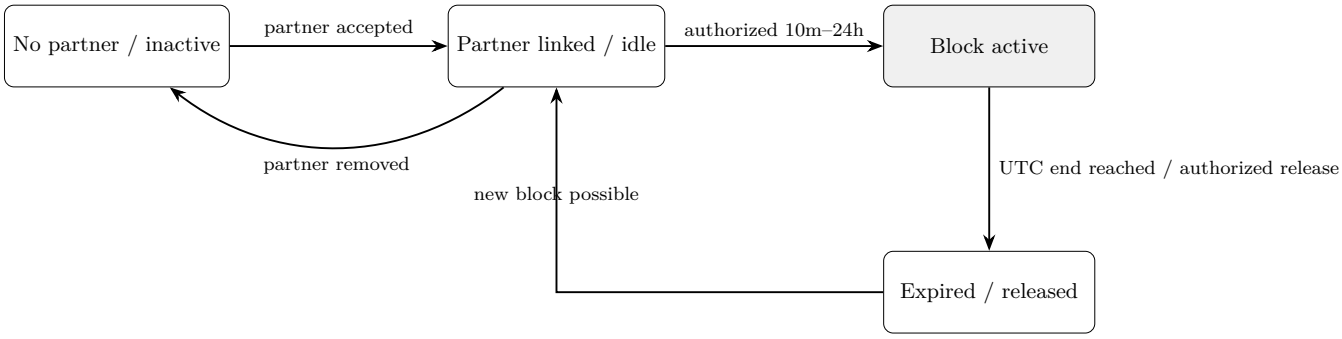


Figure 2: Proposed TradeIO-local block lifecycle. A broker remains unaffected.

Figure legend: rounded rectangles = states; gray = active restriction; arrows = allowed transitions with guard/event labels, not data movement. A declined link remains in the unlinked state; expired restrictions cease when server clock reaches end time even if no cleanup job runs.

3. Edge Cases and Error Handling

Each component has at least one explicit example for all four class categories: **B** boundary, **T** timing/concurrency, **I** bad/missing input, and **E** external dependency. **R** means recoverable after user action/retry; **N** means this operation fails definitively, though a later corrected request may succeed. These are proposed decisions for team review.

ID	Component	Category	Edge case	Decided behavior; named handler; recoverable?
EC01	C1 UI	B	Empty dashboard period	Show empty state; `DashboardView`; R
EC02	C1 UI	T	Double-click trade submit	Reuse idempotency key, disable button; `TradeForm`; R
EC03	C1 UI	I	Invalid numeric entry	Show field error; `TradeFormValidator`; R
EC04	C1 UI	E	API request times out	Keep draft, offer retry; `ApiClient`; R
EC05	C2 API	B	Exactly at daily trade cap	Apply configured inclusive threshold; `RiskGate`; N
EC06	C2 API	T	Two submissions race	Serialize authoritative cap check; `TradeController`; R
EC07	C2 API	I	Account ID belongs to another user	403, no data; `OwnershipGuard`; N

ID	Component	Category	Edge case	Decision / handler / recovery
EC08	C2 API	E	Identity service cannot validate	503, block mutation; `AuthMiddleware`; R
EC09	C3 Processing	B	Quantity exactly zero	400 reject; `TradeNormalizer`; R
EC10	C3 Processing	T	Duplicate import or replay	Return existing record / no second write; `IdempotencyGuard`; R
EC11	C3 Processing	I	Missing instrument or bad timestamp	400 structured fields; `TradeNormalizer`; R
EC12	C3 Processing	E	Contract metadata missing	Mark P&L unverified, retain raw record when safe; `InstrumentAdapter`; R
EC13	C4 Analysis	B	No closed trades / zero denominator	Zero count, win rate N/A; `BehaviorAnalyzer`; R
EC14	C4 Analysis	T	Late imported trade changes prior week	Invalidate affected snapshot, recompute; `AnalysisJob`; R
EC15	C4 Analysis	I	Invalid time zone or period	400 no guessed boundaries; `RiskAnalyzer`; R
EC16	C4 Analysis	E	AI provider offline	Deterministic summary only; `InsightAdapter`; R
EC17	C5 Storage	B	Largest supported DECIMAL value exceeded	Reject before persistence; `DbRepository`; R
EC18	C5 Storage	T	Transaction conflict / deadlock	Bounded retry with same key; `TransactionRunner`; R
EC19	C5 Storage	I	Missing FK to account	Rollback and map not found; `DbRepository`; R
EC20	C5 Storage	E	Neon connection outage	503; no success claim; `DatabaseClient`; R
EC21	E1 Auth	B	Session expires exactly at submission	401, require re-login; `SessionGuard`; R
EC22	E1 Auth	T	Partner permission revoked during block request	Recheck permission at write; `PermissionGuard`; R
EC23	E1 Auth	I	Missing/forged token	401 reject; `SessionGuard`; N
EC24	E1 Auth	E	Clerk unavailable	Fail closed for protected mutation; `AuthAdapter`; R
EC25	E2 AI	B	Empty weekly summary	Do not generate invented insight; `InsightJob`; R
EC26	E2 AI	T	Slow provider exceeds budget	Timeout; core analysis continues; `InsightAdapter`; R
EC27	E2 AI	I	Malformed provider result	Discard output, display no insight; `InsightValidator`; R
EC28	E2 AI	E	Rate limit or outage	Defer; bounded retry; `InsightQueue`; R
EC29	E3 Notify	B	User opted out	Skip optional external delivery; `NotificationPolicy`; R
EC30	E3 Notify	T	Provider confirms after timeout	Deduplicate and reconcile; `OutboxWorker`; R
EC31	E3 Notify	I	No verified destination	Mark undeliverable, in-app only; `RecipientValidator`; R

ID	Component	Category	Edge case	Decision / handler / recovery
EC32	E3 Notify	E	Provider 5xx	Backoff retry, preserve event; `OutboxWorker`; R
EC33	Config	B	Block duration 10m or 24h	Accept inclusive bounds; `BlockPolicy`; R
EC34	Config	T	Risk rule updated mid-request	Compare settings version in transaction; `SettingsService`; R
EC35	Config	I	Invalid IANA zone/currency	400, retain previous values; `PreferencesValidator`; R
EC36	Config	E	Time-zone library data missing	Reject affected conversion, store UTC safely; `TimeService`; R
EC37	Audit/log	B	Long message contains sensitive text	Truncate/redact; `AuditSanitizer`; R
EC38	Audit/log	T	Repeated override action	Stable event key prevents duplicate audit; `AuditWriter`; R
EC39	Audit/log	I	Missing actor identity	Reject privileged event; `AuditGuard`; N
EC40	Audit/log	E	Telemetry sink down	Use protected local log/outbox; `AuditWriter`; R
EC41	Broker	B	No trades on sync	Successful empty sync; `BrokerSyncWorker`; R
EC42	Broker	T	Overlapping sync jobs	Lease per connection and upsert; `BrokerSyncWorker`; R
EC43	Broker	I	Unknown instrument multiplier	Flag unverified figures, do not guess; `BrokerNormalizer`; R
EC44	Broker	E	Token revoked / API offline	Reconnect or retry; manual entry works; `BrokerAdapter`; R
EC45	Block	B	Duration below 10m or over 24h	400 reject; `BlockPolicy`; R
EC46	Block	T	Time elapses while user active	Server checks UTC expiry each protected call; `BlockGuard`; R
EC47	Block	I	No linked partner / unauthorized actor	403 reject; `BlockGuard`; N
EC48	Block	E	Notification provider unavailable	Block saved; alert remains in-app; `BlockService`; R

Failure semantics: *Retry* applies only to transient and idempotent operations; *degrade* applies only to optional AI, notification, or non-critical display; *fall back* permits manual trades when broker import is unavailable; *surface error* applies to invalid requests, denied permissions, and uncertain critical writes. Logging never overrides privacy boundaries.

4. Requirements Traceability Table

Important: Requirement IDs below are provisional D2 IDs built from the features described in the supplied D0/D1 and later stated TradeIO scope. They must be checked against the final approved `User_Stories.md`; they are *not* represented as the original story IDs. Priorities are proposals (M = must, S = should, C = could). Every mapping names a specific entity, endpoint, algorithm, figure or edge-case row rather than just a component.

ID / type	Short requirement	Pri.	Design element, mechanism and edges
F01 F	Account sign-in and access protection	M	`requireUser`; Clerk I6, `OwnershipGuard`; EC21–24
F02 F	Multiple trading accounts	M	`TRADING_ACCOUNT.user_id` FK; D1 Fig.1; EC07
F03 F	Manual trade entry	M	`POST /api/v1/trades`; `processTrade`; D2 Fig.1; EC02,09–11
F04 F	Trade history and filters	M	`GET /api/v1/trades`; D1 `TRADE(account_id,opened_at)` index; EC01,20
F05 F	Daily/weekly journal and psychology	M	`POST /api/v1/journal`; `JOURNAL_ENTRY`; D1 Algorithm B; EC11,13
F06 F	Deterministic performance analysis	M	D1 Algorithm B, `ANALYSIS_RECORD`; `GET .../analysis`; EC13–15
F07 F	Account risk settings	M	`PUT .../risk-settings`; `RISK_SETTINGS`; EC05,34
F08 F	Risk/discipline flags	M	D1 Algorithm A, `evaluateRisk`; D2 Fig.1; EC05–06,15
F09 F	Dashboard and recent metrics	M	Proposed `GET .../dashboard` + D1 I7; freshness status; EC01,04,14
F10 F	Weekly/monthly goals	S	`GOAL` D1 table; dashboard goal query; EC20; goal edit API pending
F11 F	Optional AI coaching	S	`generateInsight`, I8, `INSIGHT_RUN`; EC25–28
F12 F	External/in-app risk alerts	S	`NOTIFICATION_OUTBOX`, I10; EC29–32
F13 F	Account currency INR/USD display	M	`TRADING_ACCOUNT.currency` and I9; no automatic FX; EC35
F14 F	User time-zone support	M	`USER.time_zone`; D1 Algorithm A UTC boundaries; EC15,36
F15 F	Optional brokerage import	S	`BROKER_CONNECTION`, `syncTrades`; EC41–44; G1 / scope pending
F16 F	Trader may have no accountability partner	S	`ACCOUNTABILITY_LINK` optional; D2 Fig.2; EC47
F17 F	Partner's 10m–24h temporary block	S	`ACCOUNT_BLOCK`, `BlockGuard`; D2 Fig.2; EC33,45–48; G4
F18 F	Daily trade cap and alerts	S	`RISK_SETTINGS.max_trades_per_day`, `RiskGate`; EC05–06; override policy gap
F19 F	Multi-entry/multi-exit trades	C	Proposed `TRADE_FILL`, `processTrade`; EC10,12; G2
N01 NF	Prevent cross-user data leakage	M	Session-derived subject and owner-scoped queries; EC07,23
N02 NF	Accessible web interactions	M	Semantic labeled forms, focus handling and accessible errors in §2.2; EC03–04
N03 NF	Correct decimal precision	M	D1 decimal-string DTO + decimal normalization; EC17,43
N04 NF	Correctness under concurrent writes	M	Transaction + idempotency + serialized cap evaluation; EC06,10,18
N05 NF	Availability without AI provider	M	D1 Algorithm B + I8 fallback; EC16,26–28

ID/type	Requirement	Pri.	Mapping / how / edge IDs
N06 NF	Protect credentials and personal notes	M	Clerk I6, privacy-reduced insight payload, audit redaction; EC23,37,44
N07 NF	Recover from external service outages	S	Outbox/retry and broker manual fallback; EC20,28,32,44
N08 NF	Portable, maintainable architecture	S	Versioned `/api/v1` contracts, typed service DTOs and provider adapters; EC27,32
N09 NF	Fast interactive application	S	D1 indexes `TRADE(account_id,opened_at)`; paginated `GET /api/v1/trades`; EC04,20; measurable target not fixed

Traceability gaps to resolve: No update goal is defined for F10; F15 requires approval for the supported broker and D0 decision on external systems; F17 requires scope and override policy definition; F18 requires enforcement rather than warning and the one-month cap lock; F19 does not fit into D1 single entry model; N09 has no numeric performance target. These are real issues, not empty cells. **Reverse check (architecture to requirements).**

Element	Requirement basis	Scope conclusion
C1 UI / I1,I7,I9	F03,F09,F13,F14,N02	Retain
C2 API / authorization	F01,F02,F07,N01,N04	Retain
C3 processor / I2,I3	F03,F05,F19,N03	Retain; fills pending
C4 analysis / I4,I5	F06,F08,F18,N05	Retain
C5 storage	F02–F10,N01,N04,N09	Retain
E1 Clerk / I6	F01,N01,N06	Retain
E2 AI / I8	F11,N05	Optional
E3 notification / I10	F12,N07	Optional
Preferences service	F07,F13,F14	Retain
Audit event	N01,N06	Proposed security support
Broker adapter	F15,N07	Pending provider
Block service	F16,F17	Pending policy
Insight/outbox jobs	F11,F12,N07	Optional, justified
Any new unlisted service	No requirement identified	Do not add without new story

5. Design Rationale

Status note. Each item represents a recommendation that needs to be verified by Ishaa and Samarth. The statement "Review changed" cannot be made until there are comments from peers. Refernces D0 §7 (Decision Log)

1. **Transactional trade save.** We opted for one atomic write rather than uncoordinated writes for partial records because partial records could misrepresent trading activity. We compromised on transactional simplicity in favor of correctness. Reference: D0 Central Persistence; EC18–20.

2. **Server authorization.** Session-based identity and ownership checks were used rather than client provided IDs since client can be altered. We used checks on all protected requests since privacy is very important. Reference: D0 external identity provider; EC07,21–24.

3. **Deterministic calculations.** Calculated values in C4 with AI explanations were preferred over numerical results from AI due to the need for reproducibility of thresholds. Simpler coaching content without AI was accepted for the same reason, that results should remain useful during provider unavailability. Reference: D0 core-vs-AI log; EC13–16.

4. **Currency handling.** Instead of silently adding up the INR and USD amounts, we opted for per-account currency because doing otherwise would lead to misleading figures. We tolerated the lack of consistency on multi-currency total before there is an FX source and conversion policy set in place. Reference: D1 `TRADING_ACCOUNT`; EC35.

5. **Idempotent imports.** External ID deduplication was preferred to insert on sync since providers can resend the same page. Duplicate messages mess up risk and PL stats, so we were willing to have extra syncs and keys. Reference: D0 storage rationale; EC10,42.

6. **Notification outbox.** Post-commit queued delivery was chosen over message delivery in the trade because a failure in email delivery should not undo a legitimate trade. Notification delivery was accepted on the basis of persistence being the important path. Reference: D0 notification separation; EC30–32.

7. **Time-based partner block.** Instead of depending on an unlock job alone that is scheduled to run, we opted for an expiry time that would be verified by the server itself, and not be left hanging just because of a delay in the job. Proposed feature, pending scope confirmation; EC45–48.

8. **Versioned typed API.** We preferred typed DTO objects with `/api/v1` over broad positional interfaces, because new properties must neither expose database internals nor break consumers. We agreed to maintain backwards-compatibility, as the interface was a contractual agreement. Reference: D1 API versioning; EC03,11.

Review-driven changes: None yet recorded. Following the actual review, mark affected rationale entry IDs R1–R8 with "Changed after review: PR-#" and document any necessary D0/D1 corrections explicitly, without silently altering prior submissions.

6. Peer Design Review Notes

6.1 Feedback Received

Our design for TradeIO has been reviewed by Harsh Parsana. The reviewer found the architecture easy to understand and liked the way that the system is not solely centered around keeping track of gains and losses, but focuses more on the trading discipline and behavior analysis.

The distinction between the main risk calculation and AI analysis was considered as good too.

Some issues that require clarification were found by the reviewer; these include risk configuration changes, expiration of account blockage and duplication of trades.

6.2 Peer Review Action Items

ID	Criterion	Issue / Reviewer	Planned Action	Owner	Priority / Due
PR-01	Risk	Harsh Parsana: Changing risk settings could allow users to bypass previously reached limits.	Clarify validation and when updated limits become effective.	Samarth	Must-fix; Oct. 16
PR-02	Clarity	Harsh Parsana: Temporary accountability block expiration needs clearer behavior.	Document how active sessions respond when a block expires.	Ishaa	Must-fix; Oct. 16
PR-03	Completeness	Harsh Parsana: Manual and broker-imported trades could be counted twice.	Define duplicate detection and trade reconciliation rules.	Samarth	Consider; Oct. 16

6.3 Summary of Feedback

Feedback was based on issues of risks, clarity, and completeness. The reviewer thought that our system design was well structured and gave some suggestions on situations that would impact trading responsibility or accuracy of information.

We will be able to make it clear about risk modifications, temporary blocks expiration, and avoiding duplicate transactions.

Every criterion had one action item assigned, therefore, none of the categories got more feedback.

6.4 Issues We Raised While Reviewing Another Team

Issue 1 – Feasibility

We noticed that the other team's five-minute processing budget left only 20 seconds of spare time without measured execution results.

We suggested measuring an actual preprocessing run to determine whether the timing budget is realistic.

Issue 2 – Risk

We raised a concern about results exceeding the five-minute deadline because of queue delays.

We recommended saving completed results with an over-time flag so delayed processing would not cause results to be lost.